

School of Computing and Artificial Intelligence
Faculty of Engineering and Technology
Sunway University

SEMESTER	April 2026
COURSE NAME	CSC3206 Artificial Intelligence
TITLE	Assignment 2
LECTURER	Dr. Teoh Yun Xin
PROGRAMME	Bachelor of Information Systems (Honours) (Data Analytics)
GROUP NAME	Giggle
GROUP MEMBER	1. Wong Hui San (22034540) 2. Ayu Wen Li (22017867) 3. Keertana A/P Subramaniam (23109614) 4. Siow Qi Yung (22053037) 5. Soong Shuki (23105059) 6. Vidhipriya Devi Pokiri (23057953)
SUBMISSION DATE	9 th August 2026, 23:59

Table of Contents

1	Introduction	4
1.1	Problem Description.....	4
1.2	Graph Representation	5
1.3	Assumptions	6
2	Problem Formulation	7
2.1	State Representation	7
2.2	Initial State.....	7
2.3	Goal State.....	7
2.4	Actions	7
2.5	Transition Model.....	8
2.6	Path Cost Function	8
3	A* Search Implementation Design.....	9
3.1	Algorithm Overview.....	9
3.2	Implementation Design.....	9
3.2.1	Data and Graph Construction.....	9
3.2.2	State Representation and Data Structure Design	11
3.2.3	Heuristic Function Design.....	12
3.2.4	A* Search Core Engine with Goal Test.....	14
3.2.5	Output and Integration	17
4	Modifications and Scalability Discussion	18
4.1	Cost Weight Adjustment.....	18
4.2	Dynamic Environment Changes	18
4.3	Additional Constraints	19
5	Results and Discussion	20
5.1	Results	20
5.1.1	Program Output	20
5.1.2	Visualisation	21
5.2	Discussion.....	21
5.2.1	Optimal Route Selection Analysis	21
5.2.2	Route Cost Breakdown	22
5.2.3	A* Algorithm Performance Evaluation	22

5.2.4	Limitations and Future Improvements	23
6	Conclusion	24
7	Appendices.....	25
7.1	Video Presentation and GitHub Repository.....	25
7.2	Individual Reflection	25
7.2.1	Wong Hui San (22034540).....	25
7.2.2	Ayu Wen li (22017867)	26
7.2.3	Keertana A/P Subramaniam (23109614)	26
7.2.4	Siow Qi Yung (22053037)	27
7.2.5	Soong Shuki (23105059).....	27
7.2.6	Vidhipriya Devi Pokiri (23057953).....	28
7.3	Group Discussion Evidence.....	29
7.4	AI Usage Transparency	30
	References.....	30

1 Introduction

1.1 Problem Description

This assignment project aims to model a route optimization task that addresses a house visit route planning that visits all the residential locations of all five group members with the starting location at Sunway University. The weighted graph consists of six nodes which includes the starting location and the five residential locations. Each node represents a specific location while the edges signify the possible travel routes between each location.

As the goal is to identify the most optimal route that visits all members' residential location, the overall travel cost, which includes the carbon emission generated and the travel expenses, needs to be minimised. The two factors are aggregated into the weighted cost function in which the carbon emission costs are assigned with a weight of 0.6 and travel expenses are assigned with a weight of 0.4.

The carbon emission factor was weighted higher to demonstrate the greater priority on environmental sustainability at the same time still taking financial constraint into consideration. In order to meet these constraints with the optimal route, A* search will be applied as it can efficiently search for the lowest cost route while also limiting needless exploration.

This optimization problem aligns with United Nations SDG 13: Climate Action. As iterated by Gao and Zhan (2025), the transport sector accounts for 24% of global carbon emission, contributing a significant portion to the deterioration of the environment. Consequently, encouraging optimisation of transportation systems not only helps increase performance efficiency but is also instrumental in encouraging low carbon, green transportation practices, thereby helping in the realisation of global sustainable development targets under SDG13.

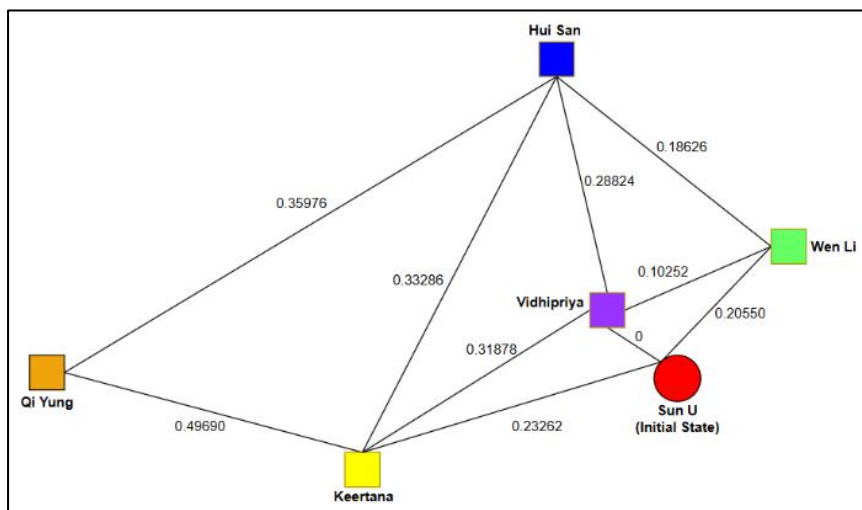
1.2 Graph Representation

Before constructing the graph, the required location and transportation data were collected in Assignment 1 using Google Maps and public transportation planning tools. Each edge in the graph was assigned a weight based on two factors, carbon emissions and travel expenses. Carbon emissions were calculated using travel distance and transportation emission factors, while travel expenses were based on fuel costs for cars and estimated fares for public transport. The two factors were then normalised and substituted into a weighted path cost function to produce cost value for each edge.

The graph representation used in Assignment 2 remains the same as the one developed in Assignment 1, as the problem formulation, locations and node connections are unchanged. The graph consists of six nodes, with Sunway University as the starting node and five other nodes representing the locations to be visited, including Vidhipriya, Wen Li, Hui San, Keertana and Qi Yung. Each node represents a distinct location and the connections between nodes represent possible travel paths.

The graph is modelled as a weighted graph, where each edge has a cost based on the carbon emissions produced and travel expenses incurred between connected locations. These weights are used by the A* search algorithm to evaluate and compare possible routes. The path cost is calculated using the formula: $\text{Path Cost} = 0.6(\text{Carbon Emission}) + 0.4(\text{Travel Expenses})$

The diagram below illustrates the graph representation used in this assignment, which serves as the search space for exploring possible routes and determining the optimal solution based on the defined edge costs.



1.3 Assumptions

To ensure the A* search algorithm focuses on finding the optimal route without the interference of highly variable real-world factors, the following assumptions were made.

- **Static Environment:** Traffic conditions are assumed to be normal (non-peak) throughout the algorithm's execution to ensure a consistent state space with constant edge weights.
- **Standardized Economic Metrics:** Fuel prices are treated as fixed based on current RON95 market rates to maintain the stability of the cost function $g(n)$.
- **Uniform Vehicle Specification:** All carbon emission and fuel consumption metrics are calculated based on a single vehicle model (Perodua Axia) to ensure parity across all nodes.
- **Data Reliability:** Distance matrices and carbon emission factors from Google Maps are accepted as ground truth to provide a deterministic foundation for the search.
- **Mathematical Consistency:** All edge weights are assumed to be non-negative to ensure the total path cost never decreases as the path grows, preventing negative cycles.
- **Normalization Stability:** The normalization method is assumed to be uniformly applicable across the 7-node graph to ensure a balanced scale for the weighted cost function.

2 Problem Formulation

2.1 State Representation

The state space is implemented as a weighted graph structure, where each residence is represented as a node and each possible route between locations is represented as an edge with a path cost. During the A* search process, each state is stored to track the progress of the algorithm so that the algorithm knows its current situation and what locations have already been visited. The current node stores the location that is currently being explored, the visited set records the residences that have already been visited to prevent repeated exploration and the path list stores the route taken from Sun U to the current location. These stored values allow the algorithm to track the route progress while searching for the optimal path.

2.2 Initial State

The initial state represents the information provided to the A* search algorithm before the route optimisation process begins. The search starts at Sunway University, which is the start node of the route and no residential locations of the group members have been visited. Therefore, the visited set is initialised as an empty set and the cumulative path cost is initialised to 0 given that no travel has occurred yet. The accumulated travel distance, transportation cost, fuel consumption, and carbon emissions are also all zero. This initial state is inserted into the A* algorithm's open list as the first search node, from which the algorithm iteratively evaluates and expands the most promising routes until all required destinations have been visited while minimising the overall travel cost.

2.3 Goal State

When the A*Search algorithms have successfully generated a route from Sunway University as the start node, and then visits all the residential locations of all the five members at least once, then the goal state is reached. The visited set of all five residential locations is as shown below:

Visited Set = {Vidhipriya, Wen Li, Hui San, Keertana, Qi Yung}

Once these residences have all been visited, the condition will be satisfied. Thus, the A* Search algorithm will stop and will return to the optimal route.

2.4 Actions

In this implementation, actions represent the discrete transitions between geographic nodes within our graph. An action is defined simply as the movement from a current location (node) to an adjacent location (node).

By abstracting these movements, we focus exclusively on the graph topology. Because the total path costs derived from carbon emissions and travel expenses are pre-calculated and embedded directly as weights on each graph edge, the algorithm no longer requires a detailed breakdown of transport modes (e.g., car vs. public transit) during the search process.

This approach streamlines the A* algorithm, allowing it to treat the problem as a standard pathfinding task. The action space is limited to the set of available edges connecting the current node to its neighbours. The algorithm evaluates these transitions based solely on the pre-computed edge weights, ensuring that the optimal sequence of actions is selected to reach the goal state while minimizing the cumulative path cost.

2.5 Transition Model

The transition model explains how the A* search algorithm moves from the current location to an unvisited residential location. During node expansion, the algorithm evaluates all neighbouring residences that have not yet been visited. Each search state is represented as (current location, visited set). When a transition is performed, the current location is updated to the new residence, and the new residence is added to the visited location. Meanwhile, the cumulative path cost, $g(n)$, will be updated by adding the travel cost which includes the transportation cost, fuel consumption, and carbon emissions. The new state is then added into the open list for further evaluation.

The general transition model is represented as:

$$(current\ location, visited\ set) \rightarrow (new\ location, visited\ set \cup \{new\ location\})$$

2.6 Path Cost Function

The path cost function calculates the cost of travelling between two connected locations. The calculated path cost is assigned as the edge weight in the weighted graph. In the A* search algorithm, this edge weight is added to the accumulated path cost, $g(n)$ when moving from the current location to the next location. The path cost prioritises carbon emissions over travel expenses, with carbon emissions given a higher weight (60%) than travel expenses (40%) to emphasise environmental sustainability while still considering travel costs. Therefore, the path cost function is calculated using $Path\ Cost = 0.6(Carbon\ Emission) + 0.4(Travel\ Expenses)$.

During the search process, A* uses this path cost function to evaluate different routes and select the route with the lowest total cost. Where:

- $g(n)$ = total accumulated path cost from Sunway University to the current location.
- $h(n)$ = estimated path cost needed to visit the remaining unvisited residences (Vidhipriya, Wen Li, Hui San, Keertana and Qi Yung).
- $f(n) = g(n) + h(n)$, which is used by A* to decide the next location to explore.

3 A* Search Implementation Design

3.1 Algorithm Overview

The A* Search algorithm is employed as the core informed search mechanism for this route optimization project, providing an efficient strategy for navigating the search space. It intelligently combines the actual path cost accumulated from the starting node, denoted as $g(n)$, with a heuristic estimate of the remaining cost required to reach the goal, represented as $h(n)$. By utilizing $g(n)$, the algorithm maintains a precise record of the path taken from Sunway University, while the heuristic function $h(n)$ provides an educated prediction of the distance to the remaining unvisited residential locations. Together, they enable the algorithm to balance the shortest known path with informed guidance toward promising solutions.

3.2 Implementation Design

3.2.1 Data and Graph Construction

```
# List of all nodes in the route planning problem
nodes = [
    "SunU",
    "Vidhipriya",
    "Wen Li",
    "Keertana",
    "Hui San",
    "Qi Yung"
]

# Raw edge data:
# Format: (From, To, Normalised Carbon Emission, Normalised Travel Expense)
raw_edges = [
    ("SunU", "Vidhipriya", 0.0000, 0.0000),
    ("SunU", "Wen Li", 0.1625, 0.2700),
    ("SunU", "Keertana", 0.0677, 0.4800),

    ("Vidhipriya", "Hui San", 0.2604, 0.3300),
    ("Vidhipriya", "Wen Li", 0.1042, 0.1000),
    ("Vidhipriya", "Keertana", 0.1313, 0.6000),

    ("Wen Li", "Hui San", 0.1771, 0.2000),

    ("Keertana", "Hui San", 0.2281, 0.4900),
    ("Keertana", "Qi Yung", 0.1615, 1.0000),

    ("Hui San", "Qi Yung", 0.2396, 0.5400)
]
```

To represent the connections between Sunway University and the five residential locations, the implementation begins by defining a list called `nodes`, which stores all six locations involved in the route planning problem including SunU, Vidhipriya, Wen Li, Keertana, Hui San, and Qi Yung. This defines the location in the graph and act as a reference of every location that exist in the A* search. It provides a simple way for the algorithm to identify the locations available during the search process and to check whether every location has been visited.

Moving on, the `raw_edges` list stores the raw travel information between connected locations. Each entry contains the starting location, destination location, normalised carbon emission value, and normalised travel expense value. These values were collected and normalised in Assignment 1 so that both criteria can be combined fairly when calculating the path cost. Only

locations that share a direct route are included in this list. This raw edge data serves as the foundation for constructing the weighted graph that will later be used by the A* search algorithm.

```
# Initialise empty adjacency list for each node
graph = {
    node: []
    for node in nodes
}

# Construct an undirected graph
# Each connection is added in both directions
for source, destination, carbon, expense in raw_edges:

    # Calculate weighted path cost
    # Cost = 0.6(Carbon Emission) + 0.4(Travel Expense)
    cost = (0.6 * carbon) + (0.4 * expense)

    # Add forward and reverse edges
    graph[source].append(
        (destination, cost)
    )

    graph[destination].append(
        (source, cost)
    )
```

After defining the raw edge data, an empty adjacency list is created for each location using a dictionary named graph. Each node from the nodes list is assigned an empty list at the beginning, which will later be filled with its connected neighbours and the corresponding path costs. This provides the structure needed to represent the weighted graph and allows the algorithm to access the available routes from each location.

Next, a loop is used to go through the raw_edges data one connection at a time. For each connection, the carbon emission and travel expense values are extracted and combined to calculate the weighted path cost. The cost is calculated by assigning a weight of 60% to carbon emission and 40% to travel expense. Since the route is bidirectional, this calculated cost is appended to both the source and destination nodes at the same time. This ensures the graph is fully undirected, meaning travel between any two connected locations costs the exact same in either direction.

```
# Display adjacency list with costs rounded to 5 decimal places
print("Graph Adjacency List:\n")

for node in graph:
    formatted_edges = [
        (neighbour, round(cost, 5))
        for neighbour, cost in graph[node]
    ]

    print(f"{node}: {formatted_edges}")
```

Finally, the completed graph is displayed using a loop that goes through each node in the graph dictionary. For every location, the code collects its neighbouring nodes and their corresponding path costs, then rounds the cost values to 5 decimal places to make the output easier to read. The formatted adjacency list is then printed, showing each location together with its available connections and calculated costs. This step is mainly used to verify that the graph has been constructed correctly before the data is passed into the A* search algorithm.

3.2.2 State Representation and Data Structure Design

```
# Define the state structure used by A* Search.
# Each state stores the current location, visited nodes, accumulated cost g(n), and the route taken so far.

class State:

    def __init__(self, current_node, visited_set, g_cost, path):

        # Current node being explored
        self.current_node = current_node

        # Nodes that have already been visited
        self.visited_set = visited_set

        # Accumulated path cost from the starting node
        self.g_cost = g_cost

        # List of nodes representing the route history
        self.path = path

    # Display state information in readable format
    def __repr__(self):

        return (
            f"State("
            f"Node={self.current_node}, "
            f"Visited={self.visited_set}, "
            f"g(n)={self.g_cost:.5f}, "
            f"Path={self.path})"
        )
```

The State class is created to represent the current progress of the A* search process. Each state stores important information needed by the algorithm, such as the current location, the path taken and the cost of travelling. Using a class helps organize all related information together to make the code easier to manage.

The `__init__()` method is used to create a new state and assign its information. It stores four main details including the `current_node`, which shows the location currently being explored, the `visited_set`, which keeps track of locations that have already been visited, the `g_cost`, which records the actual cost travelled from the starting point and the `path`, which stores the route taken so far. These details help A* decide which path to explore next.

The `__repr__()` method decides the format of the output when printing a State object. It shows the current node, visited locations, travel cost and route in a clear format instead of displaying a confusing object reference. This makes it easier to check the search progress and confirm that the stored information is correct.

```

# Create the initial state
# Starting point: Sunway University

initial_state = State(
    current_node="SunU",
    visited_set={"SunU"},
    g_cost=0,
    path=["SunU"]
)

# Display initial state
print(initial_state)

```

The `initial_state` object represents the starting point of the A* search, which is Sunway University, SunU. Since the algorithm has not moved to any nodes yet, the accumulated cost `g_cost` is set to 0, the visited set contains only "SunU" and the path consists of the starting location. This initial state serves as the first state that the A* algorithm will explore.

The `print(initial_state)` statement shows the details stored in the starting state. It helps to check whether the initial state has been created correctly before the A* algorithm starts. The output displays the starting location, visited nodes, travel cost and current route to confirm that the algorithm is ready to begin searching.

3.2.3 Heuristic Function Design

```

# Define the heuristic function h(n).
# The heuristic estimates the remaining cost by finding the minimum cost to an unvisited neighbouring node.

def heuristic(node, visited):

    # Store possible costs to unvisited neighbours
    possible_costs = []

    # Check all neighbouring nodes
    for neighbour, cost in graph[node]:

        # Ignore nodes that have already been visited
        if neighbour not in visited:
            possible_costs.append(cost)

    # If all connected nodes have been visited, no remaining cost is required
    if not possible_costs:
        return 0

    # Return the minimum estimated remaining cost
    return min(possible_costs)

```

Firstly, the `heuristic()` function was created to obtain the estimation of the remaining cost from the current during the A*Search process. The function receives information in two pieces. The pieces are the node and visited set. The current node indicates the location that is being explored. Meanwhile, the visited set records the locations that were already visited. This helps the function avoid considering the nodes that have already been visited or included in this route.

The possible_costs list was used to store the neighbouring nodes that were not visited yet edge costs temporarily. The for loop was used to check every neighbouring node that is connected to the current node by using the graph[node]. The code checks if each neighbour is not included in the visited set. The edge cost of the corresponding neighbour that has not been visited yet is added into the possible_costs list.

Once the neighboring nodes are checked, the function will determine whether the possible_costs list is empty. The function will return 0 if there are no unvisited neighbouring nodes available. This means that no immediate remaining cost can be estimated from the current node. However, the function will return the minimum value stored in possible_costs if there are unvisited neighbouring nodes available. Thus, the cheapest immediate cost of moving from the current node to one of the unvisited neighbouring nodes is represented by the heuristic value.

```
# Test heuristic calculation for the starting node

print(
    "h(SunU):",
    round(
        heuristic("SunU", {"SunU"}),
        5
    )
)
```

The code above tests the heuristic function using Sunway University indicated as “SunU”, as the starting node. Since the search has not moved to any other locations yet, the visited set only contains “SunU”. The lowest available edge cost will be returned when the function checks all the unvisited neighbouring nodes connected to Sunway University.

The round() function displays the heuristic value to five decimal places. Meanwhile, the print() statement labels the result as h(SunU). This can help the test confirm whether the heuristic function is working correctly before it can be used inside the A*Search algorithm. The output h(SunU): 0.0 represents the minimum cost which is zero from Sunway University to an unvisited neighboring node based on the graph data.

3.2.4 A* Search Core Engine with Goal Test

```
import heapq

def a_star_search():

    # Open list stores unexplored states using priority queue
    # States with the lowest f(n) are expanded first
    open_list = []

    # Closed list stores already explored states
    closed_list = set()

    # Create initial state at Sunway University
    start = State(
        current_node="SunU",
        visited_set={"SunU"},
        g_cost=0,
        path=["SunU"]
    )

    # Calculate initial f(n) = g(n) + h(n)
    f_start = (
        start.g_cost +
        heuristic(
            start.current_node,
            start.visited_set
        )
    )

    # Add starting state into Open List
    heapq.heappush(
        open_list,
        (f_start, start)
    )
```

This is the core engine of the A* search algorithm, implemented using a priority queue to store and select routes based on the lowest estimated total cost. The Open List uses Python's `heapq` module, which provides a min-heap structure where the smallest value has the highest priority, allowing the algorithm to expand the route with the lowest $f(n)$ cost first (Heaps, 2025).

At the beginning of the function, an empty `open_list` and `closed_list` are created. The `open_list` stores all possible states that have not been explored, while the `closed_list` keeps track of states that have already been visited to prevent repeated searches. The starting state is initialized at Sunway University (SunU) using the `State` class, with SunU as the starting location, an initial cost of 0, and the initial path recorded.

After creating the starting state, the initial $f(n)$ value is calculated using $f(n) = g(n) + h(n)$, where $g(n)$ is the current travel cost and $h(n)$ is the estimated remaining cost from the heuristic function. The starting state is added to the `open_list` using `heapq.heappush()` so the lowest $f(n)$ state can be selected first.

```

# Continue searching until Open List is empty
while open_list:

    # Select state with the lowest f(n)
    current_f, current = heapq.heappop(
        open_list
    )

    # Create unique state identifier
    # based on current location and visited nodes
    state_key = (
        current.current_node,
        tuple(sorted(current.visited_set))
    )

    # Skip states that have already been explored
    if state_key in closed_list:
        continue

    # Add current state into Closed List
    closed_list.add(state_key)

    # Goal test: Stop when all locations have been visited
    if current.visited_set == set(nodes):
        return current

```

The main searching process is controlled by a while loop that continues running as long as there are available states inside the `open_list`. During each iteration, `heapq.heappop()` selects the state with the lowest $f(n)$ value from the `open_list`, which becomes the next state to be explored. Before exploring the state, a unique `state_key` is created using the current location and visited locations to check whether the state has already been explored in the `closed_list`. If the state has already been visited, it will be skipped to avoid unnecessary repeated calculations. Otherwise, the state is added into the `closed_list`.

The algorithm then performs a goal test by checking whether the current `visited_set` contains all required locations in the graph. If all nodes have been visited, the current state is returned as the final solution. If the goal has not been reached, the algorithm continues by exploring all neighbouring locations connected to the current node through the graph.

```

# Explore neighbouring nodes
for neighbour, cost in graph[current.current_node]:

    # Prevent revisiting previously visited nodes
    if neighbour in current.visited_set:
        continue

    # Create updated visited set
    new_visited = current.visited_set.copy()
    new_visited.add(neighbour)

    # Create updated route path
    new_path = current.path.copy()
    new_path.append(neighbour)

    # Calculate new accumulated cost g(n)
    new_g = current.g_cost + cost

```

For each neighbouring node, the code first checks whether the location has already been visited. This prevents the algorithm from returning to previously explored locations and creating unnecessary loops. If the neighbour is a new location, the algorithm updates the visited locations by adding the neighbour, extends the current path with the new location, and calculates the new g_cost by adding the travel cost from the current node to the neighbour.

```

# Create new search state
new_state = State(
    current_node=neighbour,
    visited_set=new_visited,
    g_cost=new_g,
    path=new_path
)

# Calculate f(n) = g(n) + h(n)
f_cost = (
    new_g +
    heuristic(
        neighbour,
        new_visited
    )
)

# Add new state into Open List
heapq.heappush(
    open_list,
    (f_cost, new_state)
)

# Return None if no valid route is found
return None

```

A new search state is then created using the updated location, visited locations, total cost, and route path. The new $f(n)$ value is calculated by adding the updated g_cost and heuristic value $h(n)$. This new state with its calculated $f(n)$ value, is inserted into the `open_list` using `heapq.heappush()`. The priority queue automatically arranges the states so that the route with the lowest estimated total cost will be selected first in the next iteration. The search repeats this process until a route that visits all locations is found. If the `open_list` becomes empty before reaching the goal, the algorithm returns `None`, indicating that no valid route exists.

3.2.5 Output and Integration

```
# Display the final A* search result
def display_result(result):

    # Check whether a valid route was found
    if result is None:

        print("No route found")

    else:

        # Display final route
        print("Optimal Route:")
        print(
            " → ".join(result.path)
        )

        # Display all visited locations
        print("\nVisited Nodes:")
        print(
            result.visited_set
        )

        # Display total accumulated path cost
        print("\nTotal Cost:")
        print(
            round(
                result.g_cost,
                5
            )
        )

# Main program execution
# Run A* Search and display the result

result = a_star_search()

display_result(result)
```

The `display_result` function serves as the critical interface between the algorithm's backend processing and the user, responsible for interpreting and presenting the final route data. It first performs a validation check to ensure the search was successful; if the algorithm returns `None`, it signals that no valid path exists. Upon a successful search, the function systematically parses the result object to display the optimal sequence of locations, joined into a clear, readable string followed by the set of all unique visited nodes.

The final part of the code connects the search logic with the output display through the main program section. By assigning the search outcome to the variable `result` via a call to `a_star_search()`, the program completes the pathfinding process. Passing the result to `display_result()` completes the process by showing the optimal path, visited residences, and total path cost rounded to five decimal places.

4 Modifications and Scalability Discussion

4.1 Cost Weight Adjustment

The cost weights determine the importance of carbon emissions and travel expenses when calculating the path cost. In this assignment, since the main objective is to prioritise sustainability, carbon emissions are given a higher weight than travel expenses. This means the route planning gives higher priority in reducing carbon emissions while still considering travel expenses but lower priority than carbon emissions. Therefore, the path cost function is $0.6(\text{Carbon Emission}) + 0.4(\text{Travel Expense})$.

Adjusting the weights only changes how the path cost is calculated. For example, if the objective is to focus more on travel expenses, the travel expense weight can be increased, causing A* algorithm to prefer routes with lower travel costs. Similarly, if the carbon emission weight is increased, A* algorithm will prioritise routes with lower carbon emissions. Therefore, different weight combinations may result in different optimal routes.

However, adjusting the cost weight does not affect the A* algorithm. It still follows the same search process by calculating the path cost, updating $g(n)$, calculating $f(n) = g(n) + h(n)$ and choosing the route with the lowest $f(n)$ for further exploration. The only difference is that the path cost values are different because the weights have changed.

Although the A* algorithm remains unchanged, adjusting the weights changes the path cost calculation and may result in different optimal routes. The table below shows how different weight combinations affect the selected route. The highlighted path cost function is used in this assignment, as it places greater emphasis on reducing carbon emissions while still considering travel expenses. The other two combinations represent alternative preferences based on different decision criteria.

Path Cost Function	Priority	Possible Route Preference
$0.6(\text{Carbon Emission}) + 0.4(\text{Travel Expense})$	Sustainability	Lower carbon emission route
$0.4(\text{Carbon Emission}) + 0.6(\text{Travel Expense})$	Expenses	Cheaper route
$0.8(\text{Carbon Emission}) + 0.2(\text{Travel Expense})$	Sustainability	Much lower carbon emission route

4.2 Dynamic Environment Changes

In the real world, environment conditions may easily change over time, which make the previously calculated optimal path no longer the best option. In this implementation, edge weights are calculated based on a weighted combination of carbon emissions and travel expenses. Therefore, these changes can be handled by using the same path cost calculation formula but update the cost values based on the latest conditions.

For example, traffic congestion is a very common environmental change. It can increase travel time, and longer travel times result in higher fuel consumption, leading to increased carbon

emissions and travel expenses. Another example is changes in fuel prices or toll fees, which can directly affect travel expenses. These changes require the corresponding edge weights in the graph to be updated before running the A* search algorithm again.

Certain changes like road closures or unavailable routes, can make one location no longer be directly reached from another location. These changes may not directly affect carbon emissions or travel expenses. Instead, the corresponding edge can be removed from the graph to represent the unavailable route.

After the graph is updated, the A* algorithm must be re-executed to generate a new optimal path. The existing A* implementation is reusable and does not require structural changes to support these dynamic environments. However, it is not a real-time adaptive system because previously generated solutions will not automatically update when conditions change. To make the system continuous adaptation to changing conditions, additional features such as real-time updates and automatic route recalculation would be required.

4.3 Additional Constraints

Additional constraints may be added depending on what users prioritise, how they wish to define an optimal route, and the SDGs they aim to support. One possible constraint is budget limitations in which the total travel costs may be limited to a certain amount. In cases such as these, the algorithm would be required to check whether the accumulated travel expense remains within the allowed budget before accepting a solution. The goal test may be modified to also verify that the total travel expense satisfies the budget constraint. Paths that exceed the budget can also be discarded during the search to reduce unnecessary exploration.

Another constraint is time whereby the travel journey must be completed with a certain timeframe or duration. Therefore, travel time will need to be included as a new cost dimension. The state representation could be extended to store the total travel time together with the current location and visited locations. The cost function may also be modified to also include travel time together with carbon emissions and travel expenses, depending on the optimisation objective.

Distance constraints may be introduced to limit the total travelling distance or the distance between each state. To deal with this constraint, the A* search algorithm may validate each new state before adding it to the open list. If moving to a new location causes the total distance to exceed the specified limit, that path can be pruned and excluded from further expansion.

The additional constraints may require extending the state representation, cost function, or goal test condition to ensure that all requirements are satisfied. However, the main A* search process may be unchanged as the algorithm allows additional constraints to be added while still maintaining the basic A* structural framework.

5 Results and Discussion

5.1 Results

5.1.1 Program Output

The A* search algorithm was successfully executed. The program output verified that the search completed without errors, effectively navigating the state space to derive the optimal sequence of travel. The execution of the script produced a clear, deterministic output representing the optimal path and the final cost metrics. The algorithm successfully identified the path starting from Sunway University and sequentially visiting all member residences, concluding the search process once the goal state was reached. The terminal output confirms that the system processed all graph nodes successfully, verifying the integrity of the adjacency list and the heuristic function logic.

```
*** Graph Adjacency List:
```

```
SunU: [('Vidhipriya', 0.0), ('Wen Li', 0.2055), ('Keertana', 0.23262)]
Vidhipriya: [('SunU', 0.0), ('Hui San', 0.28824), ('Wen Li', 0.10252), ('Keertana', 0.31878)]
Wen Li: [('SunU', 0.2055), ('Vidhipriya', 0.10252), ('Hui San', 0.18626)]
Keertana: [('SunU', 0.23262), ('Vidhipriya', 0.31878), ('Hui San', 0.33286), ('Qi Yung', 0.4969)]
Hui San: [('Vidhipriya', 0.28824), ('Wen Li', 0.18626), ('Keertana', 0.33286), ('Qi Yung', 0.35976)]
Qi Yung: [('Keertana', 0.4969), ('Hui San', 0.35976)]
```

```
*** Optimal Route:
```

```
SunU → Vidhipriya → Wen Li → Hui San → Keertana → Qi Yung
```

```
Visited Nodes:
```

```
{'Keertana', 'Qi Yung', 'Hui San', 'SunU', 'Wen Li', 'Vidhipriya'}
```

```
Total Cost:
```

```
1.11854
```

The optimal route determined by the algorithm is:

SunU → Vidhipriya → Wen Li → Hui San → Keertana → Qi Yung

This sequence represents the most efficient path through the network, ensuring that the algorithm visits every node, Sunway University and the five member residences exactly as required.

The total accumulated path cost for the optimal sequence is **1.11854**. This result is calculated based on the path cost formula: $\text{Path Cost} = 0.6(\text{Carbon Emission}) + 0.4(\text{Travel Expenses})$. This final value combines both the carbon footprint and the travel cost of the journey, illustrating the algorithm's success in balancing environmental sustainability with financial constraints.

5.1.2 Visualisation

Link: https://drive.google.com/file/d/1HQfuByNIHX_1GSTJhUk10Ev-Bn4kdKnY/view?usp=sharing

The video visualization shows the movement from Sunway University to Vidhipriya, Wen Li, Hui San, Keertana, and Qi Yung in sequence as it demonstrates the final optimal route produced by the A*Search algorithm. The completed route for each segment is visible and the markers indicate the locations that have already been visited. This helps the viewer to understand the travel order clearly and progression of the route until the goal is reached.

Python was used to develop the video. The six locations were first defined using the latitude and longitude of the locations. Then the Open Source Routing Machine (OSRM) was used to obtain the driving route layout between each pair of the consecutive locations. This is to ensure that the path is animated following the road network. CARTO map tiles were used to generate the map background based on the OpenStreetMap data. Meanwhile, Pillow was utilized to draw the route lines, location markers, and moving nodes on each frame. The actual road route between each location is shown by the moving node. In addition, the completed route segments remain highlights so that the visited locations and route are marked clearly.

5.2 Discussion

5.2.1 Optimal Route Selection Analysis

The A* algorithm selected the optimal route as SunU → Vidhipriya → Wen Li → Hui San → Keertana → Qi Yung with a total accumulated path cost of 1.11854. The path cost was calculated using $\text{Path Cost} = 0.6(\text{Carbon Emission}) + 0.4(\text{Travel Expenses})$. This formula allows the algorithm to consider both sustainability and financial factors when selecting the best route. Since carbon emission has a higher weight compared with travel expenses, the algorithm gives higher priority to routes with lower carbon emissions.

During the A* search process, the algorithm evaluated all possible routes based on their total accumulated cost. Routes with higher carbon emissions or higher travel expenses produced a higher path cost and were not selected. Therefore, the selected route was chosen because it had the lowest overall path cost while balancing sustainability and reduced travel expenses.

For example, the connection from Vidhipriya to Wen Li had a carbon emission value of 0.1042 and a travel expense value of 0.1. Using the path cost function, this connection produced a lower cost compared with the alternative connection from Vidhipriya to Hui San, which had higher carbon emission (0.2604) and travel expense (0.3300). Although Vidhipriya to Hui San was also able to continue the journey, this connection produced a higher cost because it created a higher impact on both sustainability and travel expenses. Therefore, A* selected Wen Li because it had a lower total cost and provided a more sustainable route towards the destination.

For every connection between two locations, the A* algorithm calculates the edge cost using the carbon emission and travel expense values. By accumulating the cost of each selected edge, the algorithm identifies the route with the lowest total cost while maintaining a focus on sustainability.

5.2.2 Route Cost Breakdown

Within the collected route information, the most expensive segment is from Keertana to Qi Yung, with a cost of 0.4949. This cost is noticeably higher than that of the other segments. Although it has the highest overall cost, it is not caused by carbon emissions alone, as its carbon emission value is only 0.1615, which is lower than some other segments. Instead, the high cost is mainly driven by its travel expense of 1.0, the highest among all segments. This suggests that the Keertana and Qi Yung route is less likely to be selected despite not being the most polluting route.

The second most expensive segment is Hui San and Keertana, with a cost of 0.3329. Unlike the previous segment, its cost is influenced by a more balanced combination of carbon emissions and travel expenses, suggesting a relatively longer travel distance overall.

On the other hand, the segment from SunU to Vidhipriya has a cost of 0, indicating that the two nodes are extremely close to each other. As a result, both the travel expense and carbon emissions are minimal.

Overall, these results show that the path cost is not determined by distance alone. A route may have relatively low carbon emissions but still produce a high overall cost due to travel expenses, or vice versa. This reflects how the cost formula balances both environmental and financial factors when evaluating different routes.

5.2.3 A* Algorithm Performance Evaluation

A* search uses a heuristic function to estimate the remaining cost from each state and guide the search towards the most promising route. This allows A* to explore fewer unnecessary paths while still identifying the optimal solution. The efficiency comes from its evaluation function, $f(n)$, combining the actual cost accumulated from the starting point, $g(n)$, and the estimated cost required to reach the goal, $h(n)$. By prioritising states with lower $f(n)$ values in the open list, the algorithm expands more promising paths, instead of blindly exploring states based on their generation order. This is especially beneficial in this problem, as a brute-force approach for five residences could require evaluating up to 120 possible visiting sequences. In contrast, A* can eliminate less favourable paths early when the estimated total cost shows that they are unlikely to produce a better solution.

A* is suitable for this route optimisation problem because it is designed to find the lowest-cost path by considering both the current cost and the estimated future cost. In this problem, each

connection between Sunway University and the residential locations has different carbon emission and travel expense values. These factors are combined into a weighted path cost, allowing A* to evaluate routes based on both environmental impact and travel expense. Therefore, the algorithm is able to select a route that provides the lowest overall cost instead of only focusing on one factor.

Additionally, the closed list improves the search efficiency by preventing the same state from being expanded multiple times. Explored states are stored in the closed list to avoid unnecessary repeated searches. Overall, the combination of heuristic guidance and state tracking enables A* to efficiently find the optimal route while reducing unnecessary exploration compared to other search method.

5.2.4 Limitations and Future Improvements

Limitations

The current A* search algorithm has several limitations that reduce its ability to accurately represent real-world or handle route planning scenarios. Firstly, the main limitation is the simplified heuristic function whereby the current heuristic function denoted by $h(n)$ estimates the remaining cost by using the minimum edge cost from the current node to the unvisited neighbour node. However, this approach gives only a weak estimation of the actual remaining travel cost as it only considers the immediate connections instead of the overall cost of visiting remaining locations.-This may cause a need in larger search space and higher computational needs when being implemented on networks with larger sizes.

Additionally, using a fixed cost model when calculating travel costs and carbon emissions depends on the current system making assumptions about the constant value of fuel prices, emissions and travel conditions. However, the assumptions are unrealistic as they do not reflect real-world changes from factors such as traffic, weather, fuel prices or driving behaviours. Moreover, the route network is developed using predefined nodes and edges that may also not fully represent real road networks where there are more alternative options for routes or roads.

The third limitation is the state representation, as the current state class only stores the current location, visited location, total cost and route taken, without considering additional factors such as time constraints, budget limitations or fuel availability. Thus, the current system may not be applied to real world route planning situations.

Future Improvements

To address the current system's limitations, several improvements can enhance its flexibility and real-world applicability. In regards to the simplified heuristic function limitation, it can be improved by including more advanced methods such as nearest-neighbour heuristics or minimum spanning tree. This helps the algorithm to have more accurate estimation of the remaining travel cost, improving efficiency and scalability for larger networks.

Furthermore, instead of using fixed cost models, the A* search algorithms can be improved by using real-time data such as utilising live traffic, fuel prices and weather data. This allows travel costs, times and carbon emissions to be updated based on the current conditions. Besides that, the state representation can also expand to include factors such as time constraint, resources limitations and optimisations goal to improve the system's ability in handling realistic routing scenarios that includes time limits, budgets and environmental constraints.

Another possible future improvement would be to provide alternative route options instead of only one optimal route. This allows users to choose routes that best suit their preferences, enhancing the system's adaptability to changing user needs. Future work can also explore implementing machine learning to increase system intelligence and flexibility. Machine learning models can be utilised to predict travel time, calculate carbon emissions or adapt to the user's changing preference, making the system better suited for real-world route planning.

6 Conclusion

A* Search algorithm was successfully implemented in this project so that the route optimization problem that involves Sunway University and five residential locations can be solved. The locations were presented as an undirected weighted graph by the algorithm. Due to this, each route was evaluated using a cost function that included 60% of normalized carbon emissions and 40% of normalized travel expenses. When all the six nodes which include Sunway University, and the five residential locations, were visited, the goal state was achieved.

The total accumulated path cost was 1.11854 where the algorithm identified the route with minimum-cost which was SunU $\rightarrow$ Vidhipriya $\rightarrow$ Wen Li $\rightarrow$ Hui San $\rightarrow$ Keertana $\rightarrow$ Qi Yung. This finding demonstrated that A*Search is able to determine a route that can balance both environmental sustainability and financial considerations effectively. The solution produced supports SDG 13: Climate Action as it has a great emphasis on promoting the use of transportation planning with lower carbon emissions.

In summary, by using the heuristic function, the algorithm was able to focus on promising states and reduce unnecessary repeated exploration as it was enabled by the priority-based Open List and Closed List. By adding more nodes, edges, and constraints, the implementation can be extended. Thus, making it more suitable for route-planning problems on a large scale. However, strong heuristic may need to maintain the search efficiency and scalability for graphs that are substantially large.

7 Appendices

7.1 Video Presentation and GitHub Repository

The following link directs to our video presentation. The video is uploaded to Google Drive and can be viewed by anyone with the link. The GitHub repository containing all implementation files, including the source code and related project files, is also provided below for reference.

Link: <https://drive.google.com/file/d/1fCPVxKbR5VrUmulAPNvH-e4h8h2HiEJ6/view?usp=sharing>

GitHub Link: https://github.com/Keertana0307/AI_Assignment_2-Group_Giggle

7.2 Individual Reflection

7.2.1 Wong Hui San (22034540)

In this assignment, I contributed as the group leader by delegating tasks, overseeing progress, and ensuring the team stayed aligned with the project objectives. My main responsibilities included coding the A* Search algorithm, writing the Graph Representation section, explaining the A* Search Core Engine with Goal Test, and discuss code scalability in Dynamic Environment Changes.

One significant learning outcome was developing a deeper understanding of how A* search algorithms are designed and evaluated. In Assignment 1, I understood A* theoretically as an algorithm that combines $g(n)$ and $h(n)$ to find the shortest path. However, implementing A* in Assignment 2 made me realise that its effectiveness depends greatly on problem formulation and implementation design. I learned that even if an algorithm is theoretically optimal, coding and implementation choices can influence its practical effectiveness. This improved my critical thinking skills as I learned to evaluate not only the correctness of the algorithm, but also its ability to handle complex and larger problems.

The most challenging part of this assignment was designing and debugging the A* implementation. The algorithm consists of components such as the priority queue, state tracking, heuristic calculation, and route expansion, where each requires careful reasoning to ensure valid paths and correct results. Overcoming these challenges improved my understanding of how each component contributes to the overall system and strengthened my debugging skills.

As the group leader, I learned that effective teamwork requires more than just assigning tasks. It also requires monitoring progress, resolving misunderstandings, and ensuring consistency across different sections. Additionally, AI tools are used to clarify concepts, debug errors, and improve my understanding. I always critically evaluate and modify AI suggestions to align with project objectives, using AI as a support tool rather than relying on it completely.

7.2.2 Ayu Wen li (22017867)

My contributions to this project began with defining the problem in the problem description section of the report, followed by expanding the explanation of the initial state and transition model for our A* search implementation with greater technical details compared to Assignment 1. Next, I'd also analysed and documented the additional constraints related to the A* search system which included factors related to time, budget, distance and state representation. Finally, I concluded the report by discussing the possible limitations of our current implementation and suggesting possible future improvements that can be done to enhance the system.

As my contributions mostly involved critically analysing the A* search algorithm for constraints and limitations, I gained a deeper understanding of the practical considerations that are involved when implementing the algorithm with real world scenarios. Analysing these constraints and weaknesses helped me learn that the effectiveness of A* search algorithm depends on different factors such as heuristic design, state representation, computing efficiency and problem-based constraints.

As for challenges, the most difficult part of the assignment for me was identifying and analysing the additional constraints of A* search algorithm. This is because I needed to conduct extensive research to find the inherent strength and weaknesses of A* search in general and differentiate them from the weaknesses that are specific to our project.

This assignment helped improve my collaboration and team working skills as our team regularly communicated to discuss each member's task and responsibilities. My members were helpful in clarifying any questions and helped review each other's section of the project. These clarifications helped me better understand how the code was developed and what constraints the A* search model had.

7.2.3 Keertana A/P Subramaniam (23109614)

For this assignment, I contributed in explaining the Goal State and Heuristic Function Design, Conclusion, developing route visualisation video, and recording video presentation. Although my tasks are mainly on these components, I can see how the various parts of the A* Search algorithms, like graph construction, representation of the state space, computation of the heuristic, expansion of the route, and testing for a goal, interact to produce the ultimate optimal route.

I gained a much clearer understanding of A* Search through this assignment. I have understood previously that A* combines $g(n)$ and $h(n)$ where the $g(n)$ is the actual accumulated cost and $h(n)$ is the estimated remaining cost. However, in this assignment, I learnt that the quality of the final results depends on how the graph states, heuristic, and goal condition are designed.

The most difficult aspect was creating the video of the route visualisation. It involved integrating geographical coordinates, road-route information, map tiles, animation frames, and video encoding. I also needed to make sure that the route that was displayed was the same as what A* search would give me. The other challenge was in the preparation of the presentation,

ensuring that the technical explanations were simplified without compromising the accuracy of the description of the code.

This assignment also helped to enhance my teamwork and communication skills. I learnt that it is important to maintain consistency across the various sections of the report, code explanations, visualisations and presentation slides. Concepts were explained, clarified, and troubleshooting was helped with the use of AI tools. However, I did review and modify the suggestion carefully so that the output correctly matched the implementation. Overall, the assignment deepened my knowledge on A* Search, problem solving, visual communication and collaborative project building.

7.2.4 Siow Qi Yung (22053037)

In this assignment, I contributed by explaining the implementation of the A* search algorithm, including the state representation, path cost function, code implementation of the state representation, cost weight adjustment and how A* selects the optimal route. I explained how each state stores information so that the algorithm can keep track of the search process. I also explained how the state is implemented in Python using a State class and more. In addition, I discussed how the path cost is calculated using carbon emissions and travel expenses and how changing the cost weights affects the selected route without changing the A* algorithm. Finally, I discussed how A* compares different routes using the path cost to find the best route.

Our group worked together by having the group leader divide the tasks for each section, prepare the report template and set internal deadlines. We kept reviewing the assignment requirements and Assignment 1 to ensure that our problem statement and objectives were consistent. Whenever we faced any issues, we discussed them in our group chat and worked together to find a solution that everyone agreed on.

Throughout this assignment, I learned that A* is an AI search algorithm that finds the best route by searching the most suitable paths instead of checking every possible route. I also learned that state representation is important because it stores information such as the current node, visited nodes, current path and total path cost, which helps the algorithm keep track of the search process. Besides, I learned how to implement the state representation in Python. I also understood how the path cost function combines carbon emissions and travel expenses and how changing the cost weights affects the route selected based on different priorities.

7.2.5 Soong Shuki (23105059)

This project provided valuable learning experience in understanding how a search algorithm can be developed and applied using Python. One of the main lessons gained was how a graph structure can be created to represent different connections between locations and how the same graph design can be reused even when the input data changes. This demonstrates the flexibility of algorithm-based solutions in handling different scenarios without requiring the entire system to be rebuilt.

Another important learning outcome was understanding how data is prepared before being processed by the algorithm. The process of setting up the data structure, defining the relationships between nodes, and applying the required cost calculations helped improve the understanding of how raw data is transformed into meaningful information for decision-making. In addition, analysing how Python loops through possible routes to evaluate and identify the optimal solution provided a clearer understanding of the internal process behind search algorithms. On top of that, reviewing the complete code flow from the beginning to the final output had improved my ability to interpret and explain technical implementations. This experience highlighted the importance of understanding not only the final result but also the reasoning and processes that produce the result.

Beyond technical knowledge, this project also strengthened the understanding of teamwork and task management. With clear task delegation and proper progress tracking, responsibilities can be distributed effectively, making collaboration smoother and allowing issues to be identified earlier. Overall, this project improved both technical understanding of algorithm development and practical skills in communication, analysis, and teamwork.

7.2.6 Vidhipriya Devi Pokiri (23057953)

During this assignment, I participated in a variety of responsibilities that connected the planning stage of the project with its implementation. I first started by working on Assumptions and Actions, where I helped define the project's scope and the capabilities of the agent. These sections helped create a clear foundation for the rest of the project, ensuring that our A* search solution stayed focused on the problem requirements and avoided unnecessary complexity.

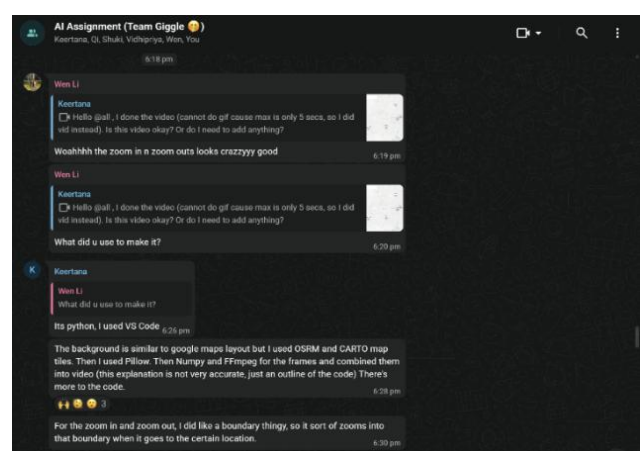
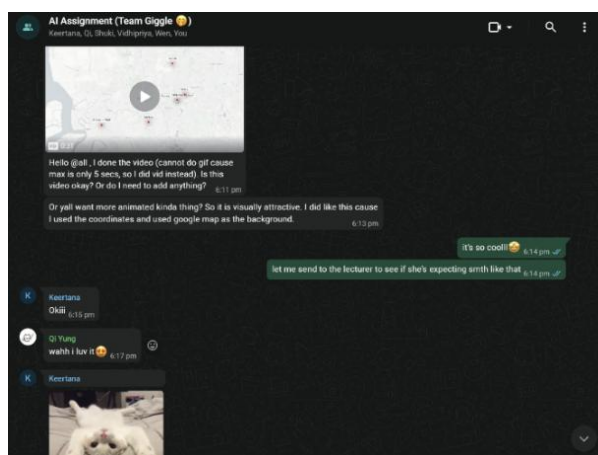
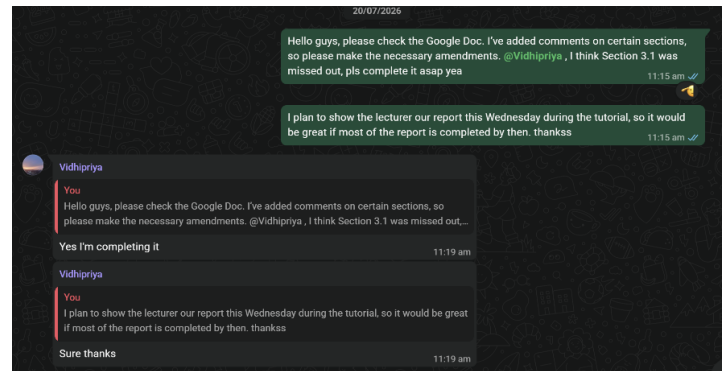
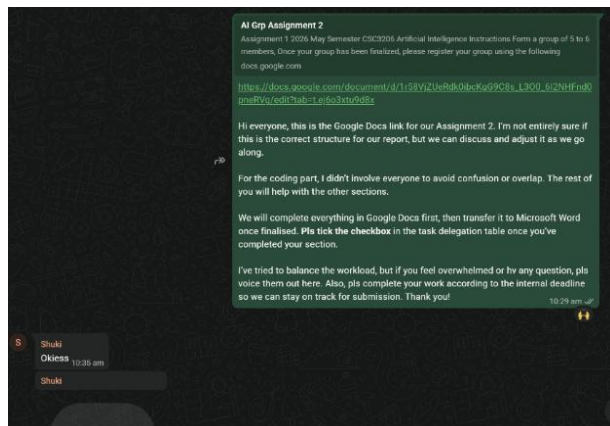
As the project progressed, I became more involved in documenting and evaluating our work. For example, when expanding on the Algorithm Overview, I explained how our A* search algorithm worked, including the heuristic and cost functions, in a way that was both technically accurate and also easy to understand. This also required alignment with my teammates to make sure the documentation matched the actual implementation. I additionally completed the Results part, where I analyzed the output to confirm that the algorithm was producing optimal and cost-efficient routes. Being able to see the algorithm perform as expected was one of the most satisfying parts of the project.

Working with my teammates was a highly valuable part of this assignment. We communicated regularly and shared feedback throughout the process, especially when refining the assumptions and documentation to match the challenges we encountered during implementation. This collaborative approach showed me how important it is to combine clear communication with technical accuracy.

Overall, this project gave me a much better understanding of how search algorithms can be applied to solve real-world problems. It also helped me appreciate the challenge of balancing multiple objectives, such as minimizing travel costs while reducing carbon emissions, within a single cost function. More so, the experience strengthened both my technical skills and additionally my ability to work effectively as part of a team.

7.3 Group Discussion Evidence

Group Discussions:



Planning documents - Task allocation records:

Task	PIC	Internal Due	Status
1.0 Introduction			☑
1.1 Problem Description	Wen Li	3/7	☑
1.2 Graph Representation	Hui San	3/7	☑
1.3 Assumptions	Vidhipriya	3/7	☑
2.0 Problem Formulation			☑
2.1 State Representation	Qi Yung	6/7	☑
2.2 Initial State	Wen Li	6/7	☑
2.3 Goal State	Keertana	6/7	☑
2.4 Actions	Vidhipriya	6/7	☑
2.5 Transition Model	Wen Li	6/7	☑
2.6 Path Cost Function	Qi Yung	6/7	☑
Coding	Hui San		☑
Visualisation	Keertana		☑
3.0 A* Search Implementation Design			☑
3.1 Algorithm Overview	Vidhipriya	13/7	☑
3.2 Implementation Design			☑
1) Data & Graph Construction	Shuki	25/7	☑
2) State Representation & Data Structure Design	Qi Yung	25/7	☑
3) Heuristic Function Design	Keertana	25/7	☑
4) A* Search Core Engine with Goal Test	Hui San	25/7	☑
5) Output & Integration	Vidhipriya	25/7	☑
4.0 Modifications and Scalability Discussion			☑

4.1 Cost Weight Adjustment	Qi Yung	15/7	☑
4.2 Dynamic Environment Changes	Hui San	15/7	☑
4.3 Additional Constraints	Wen Li	15/7	☑
5.0 Results and Discussion			☑
5.1 Results			☑
5.1.1 Program Output	Vidhipriya		☑
5.1.2 Visualisation	Keertana		☑
5.2 Discussion			☑
5.2.1 Optimal Route Selection Analysis	Qi Yung	25/7	☑
5.2.2 Route Cost Breakdown	Shuki	25/7	☑
5.2.3 A* Algorithm Performance Evaluation	Shuki	25/7	☑
5.2.4 Limitations and Future Improvements	Wen Li	25/7	☑
6.0 Conclusion	Keertana	27/7	☑
7.0 Appendices			☐
7.1 Slides & Video Presentation	Shuki	28/7	☐
	Keertana		☐
7.2 Individual Reflection	ALL	30/7	☐
7.3 Group Discussion Evidence	ALL	30/7	☐
7.4 AI Usage Transparency	ALL	30/7	☐
References	ALL	30/7	☐
Export Readme File	Hui San	31/7	☐
Final check	ALL	31/7	☐
Submit on elearn	Hui San	31/7 Actual Due: 2/8	☐

7.4 AI Usage Transparency

AI Tool: ChatGPT

- *Coding and Debugging of A Search Algorithm**
<https://chatgpt.com/c/6a68708f-8670-83ec-a445-ad43a8b123a4>
 - Used to help understand, write, and debug the A* Search algorithm. The AI explained key concepts such as state representation, graph construction, and implementation logic, while also assisting in identifying and fixing coding errors to improve the development process.
- *The state representation of code in A**
<https://chatgpt.com/share/6a58f9c4-f704-83ec-b196-b1b3c4157ec1>
 - Used to explain the state representation in the A* search implementation to help me understand how the code represents states and how the algorithm operates during the search process.
- *A Search Code Explanation for Report and Presentation*
- <https://chatgpt.com/share/6a6d659f-c758-83ec-b1d4-d976ce7f7072>
- *Grammar Review and Sentence Polishing*
- <https://chatgpt.com/share/6a6d65be-3b68-83ec-9153-98202f31a464>
 - Used to explain the A* search code step by step, which helped improve understanding of the implementation and generate ideas for the project documentation and presentation. In addition, it was used to identify grammar mistakes and polish sentences to improve writing quality

AI Tool: Gemini

- *Roles of the open list and closed list of A* Algorithm*
<https://share.google/aimode/TOKdKkujCKFIJTU3>
- *Search results of A* Algorithm*
<https://share.google/aimode/e4QxiBVXkKR1eRX4o>
 - AI assisted me in gaining a better understanding of the roles of the open and closed lists in the A* algorithm and interpreting the search results, which enabled me to clearly explain the implementation design and discuss the final findings and results.

References

- Gao, L., & Zhan, M. (2025). Route Optimization of Multimodal Transport Considering Regional Differences Under Carbon Tax Policy. *Sustainability* 2025, 17(13), 5743.
<https://doi.org/10.3390/su17135743>
- Heaps. (2025). CS106B Programming Abstractions.
<https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1262/lectures/17-heaps/>